

This is the prompt I used for the `normalize.py` program. I based the rest of the prompts I used on it. Below these are the actual prompts I used for the rest of the project.

Please identify structure aligned semantic properties from this code structure file.
Separate scopes into branch-level and function-level properties.

For example, a branch-level property would be: given an empty list as an input the function returns the same empty list. This property is branch-level because its behavior is dependent on the branches being taken.

An example of a function-level property is that the sum of the output elements will always be 1. Another function-level property is that all output values are non-negative if all inputs are non-negative.

Output the branch-level properties you identify in a JSON file with the following format:

```
{
  "scope": "branch",
  "function": name of function,
  "condition": branch condition (e.g. "len(xs) == 0"),
  "property": property shorthand (e.g. "identity_on_empty"),
  "formal": formalized property (e.g. "normalize(xs) == xs")
}
```

Output all the function-level properties you identify in a JSON file with the following format:

```
{
  "scope": "function",
  "function": function name (e.g. "normalize"),
  "property": property shorthand (e.g. "unit_sum"),
  "precondition": condition that must be true for the property to hold (e.g.
"sum(xs) != 0")
  "formal": formalized property (e.g. "sum(normalize(xs)) == 1")
}
```

(below is what Cline output)

Identified and output semantic properties for `normalize(xs)` into two files:

branch_properties.json — 4 branch-level properties:

Condition	Property
<code>len(xs) == 0</code>	identity_on_empty — returns the input unchanged

<code>len(xs) != 0 and sum(xs) != 0</code>	unit_sum_on_nonempty — output sums to 1
<code>len(xs) != 0</code>	length_preserved_on_nonempty — output length equals input length
<code>len(xs) != 0 and all(x≥0) and sum(xs)!=0</code>	nonnegative_outputs_on_nonnegative_inputs — all outputs ≥ 0

function_properties.json — 4 function-level properties:

Precondition	Property
True	length_preserved — <code>len(normalize(xs)) == len(xs)</code> always holds
<code>sum(xs) != 0</code>	unit_sum — <code>sum(normalize(xs)) == 1</code>
<code>all(x≥0) and sum(xs)!=0</code>	nonnegative_outputs — all output elements are ≥ 0
<code>sum(xs) != 0</code>	idempotent — <code>normalize(normalize(xs)) == normalize(xs)</code> (a normalized list re-normalizes to itself)

I used this prompt to generate the test cases for normalize.py:

Write tests using the Hypothesis testing framework for Python to exercise all of the semantic properties you identified earlier. These semantic properties are located in normalize/branch_properties.json and normalize/function_properties.json. Do not execute the tests.

=====

The prompts I used to generate properties and tests both followed a template. I used a shell script to repeatedly prompt Cline via CLI to generate a property file/test for each program in the dataset.

This is the base for the property generating prompt:

“

Please identify structure aligned semantic properties from the code structure file located at {FILE_PATH}. Separate scopes into branch-level and function-level properties. A branch-level property holds only under certain conditions. A function-level property holds for all executions of the function. Also identify invariants, pre-conditions, post-conditions, metamorphic relations, and any other property types that help capture the intended semantics of the program. Output the branch-level properties you identify in a JSON file with the following format:

```
{
  "scope": "branch",
  "function": name of function,
  "condition": branch condition (e.g. "len\(\xs\) == 0"),
  "property": property shorthand (e.g. "identity_on_empty"),
  "formal": formalized property (e.g. "normalize\(\xs\) == xs")
}
```

Output all the function-level properties you identify in the same JSON file with the following format:

```
{
  "scope": "function",
  "function": function name (e.g. "normalize"),
  "property": property shorthand (e.g. "unit_sum"),
  "precondition": condition that must be true for the property to hold (e.g.
"sum\(\xs\) != 0"),
  "formal": formalized property (e.g. "sum\(\normalize\(\xs\)\) == 1")
}
```

Create a JSON file in the properties directory with the same name as the structure file but with `_properties.json` suffix. For example, if the structure file is `ad_mix_structure.json`, the properties file should be `ad_mix_properties.json`. The JSON file should contain a list of all the properties you identified for that structure.

The structure of the program to analyze is:

{APPENDED_STRUCTURE_FILE}

“

This is the base for the test generating prompt:

“

Write tests using the Hypothesis testing framework for Python to exercise all of the semantic properties you identified earlier. These semantic properties are located in {FILE_PATH}. Generate your test file with the `"_test.py"` suffix in the directory called `"tests"`. Do not execute the tests.

The semantic properties of the program to analyze are as follows:

{APPENDED_PROPERTIES_FILE}

“

My scripts prompted Cline with each of these prompts once per file in the dataset.